claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_416cc0e5-a35\agent-afe11f84503f10419.jsonl`
- SHA-256: `d9c7c4f07515fd116b2db7afead8f84f8ab5c05845955dba3f0c90745013e587`
- Source modified: `2026-06-22T09:49:41+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:48:18.367Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:

```
{
 "title": "Result[List[Dict]] contract properly handled with Success/Failure unwrapping",
 "file": "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/scripts/nightly_reelcount.py:345-351",
 "severity": "low",
 "detail": "Line 349 correctly unwraps: 'segs = result.value if hasattr(result, \"value\") else result'. This handles both Success[list] (the new Result contract) and bare list returns (legacy). Works with both motion.py (returns Success) and trajectory_hybrid.py (documented as returning bare list despite declaring Result[...] in base.py).",
 "fix": "No fix needed. The contract is properly adhered to."
}
```

2. assistant (2026-06-22T09:48:20.432Z)

I'll analyze this finding by examining the code in the nightly-e2e worktree. Let me start by reading the relevant files to verify the claim.

3. user (2026-06-22T09:48:21.794Z)

```
1       """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM nightly).
2
3       Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a deployment serves
4       it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does not change**
5       vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score of CACHED
6       trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real model*, so it
```

```
needs a GPU ? hence it runs on an ephemeral GCP GPU VM (``deploy/gcp/nightly_regression/``), emails the
owner the report, and self-deletes. It is **default-additive**: a tripwire, not the promotion gate.

What it asserts / reports, per the owner's ask ("quality metrics + regression report + cost"):
  ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected highlight clips.
    Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so this is asserted
    **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-None, see §guard).
  ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match block from
    ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small tolerance.
  ? COST of the run ? VM uptime (or processing wall) × machine $/hr via ``backend.billing`` ? USD + INR.

Extensible pattern (add a video = one manifest row + re-freeze):
  ``eval_baselines/reelcount_manifest.json``  ? the videos (gs:// / local / gdrive:// URIs) + segmenter
  ``eval_baselines/reelcount_baseline.json``  ? the frozen expected counts (regenerated, never hand-edited)

Usage
-----
Freeze the baseline from the current configured+checkpointed engine (after an intended change, or first run):
    python scripts/nightly_reelcount.py --update-baseline

Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails if --email):
    python scripts/nightly_reelcount.py --email

Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality dropped) · 2 operational
(manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter changed ? re-freeze).
"""
from __future__ import annotations

import argparse
import datetime as _dt
import json
import os
import sys
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional, Tuple

REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if REPO_ROOT not in sys.path:
    sys.path.insert(0, REPO_ROOT)

# billing is pure (typing only) ? safe to import at module top.
from backend import billing  # noqa: E402

DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")

#: Quality metrics we report + (when labels exist) gate ? higher is better, small tolerance.
QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
```

```python
DEFAULT_QUALITY_TOL = 0.02     # quality is float/labeled ? a looser tripwire than the exact reel-count
DEFAULT_FX_INR_PER_USD = 83.0
#: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-south1, ~mid-2026).
DEFAULT_MACHINE_USD_PER_HR = 0.35


@dataclass(frozen=True)
class Tolerances:
    quality_tol: float = DEFAULT_QUALITY_TOL

    def to_dict(self) -> Dict[str, Any]:
        return {"quality_tol": self.quality_tol}


# --------------------------------------------------------------------------- #
# Pure cost math (wraps backend.billing ? no IO).
# --------------------------------------------------------------------------- #
def cost_report(billed_seconds: float, machine_usd_per_hr: float,
                fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
    """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing`` arithmetic).

    ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes ``--vm-uptime-seconds``;
    otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays for boot/install)."""
    rate_per_sec = float(machine_usd_per_hr) / 3600.0
    usage = {billing.WALL_SECONDS: float(billed_seconds)}
    rates = {billing.WALL_SECONDS: rate_per_sec}
    usd, breakdown = billing.compute_cost_usd(usage, rates)
    return {
        "billed_seconds": round(float(billed_seconds), 1),
        "gpu_seconds": round(float(gpu_seconds), 1),
        "machine_usd_per_hr": float(machine_usd_per_hr),
        "usd": usd,
        "inr": billing.to_inr(usd, fx_inr_per_usd),
        "breakdown": breakdown,
    }


# --------------------------------------------------------------------------- #
# Pure summary + comparison core (no IO ? unit-testable with plain dicts).
# --------------------------------------------------------------------------- #
def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
                      cost: Dict[str, Any]) -> Dict[str, Any]:
    """Compact, comparable snapshot from per-video run results.

    Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None, wall_seconds}``.
    A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it, never hides it)."""
    per_video: Dict[str, Any] = {}
    for r in run_results:
        per_video[r["name"]] = {
            "reel_count": r.get("reel_count"),
            "ok": bool(r.get("ok", False)),
            "error": r.get("error"),
            "quality": r.get("quality"),
        }
    return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
            "n": len(per_video)}
```

```python
@dataclass
class Finding:
    video: str
    metric: str                 # "reel_count" | "error" | a quality key | "segmenter" | "corpus"
    kind: str                   # "regression" | "improvement" | "stale"
    baseline: Optional[float] = None
    current: Optional[float] = None
    detail: str = ""

    def message(self) -> str:
        if self.kind == "stale":
            return f"[STALE] {self.metric}: {self.detail}"
        if self.metric == "error":
            return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
        b = "?" if self.baseline is None else f"{self.baseline:g}"
        c = "?" if self.current is None else f"{self.current:g}"
        tag = "REGRESSION" if self.kind == "regression" else "improvement"
        return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"


@dataclass
class NightlyResult:
    findings: List[Finding] = field(default_factory=list)
    added: List[str] = field(default_factory=list)
    removed: List[str] = field(default_factory=list)

    @property
    def regressions(self) -> List[Finding]:
        return [f for f in self.findings if f.kind == "regression"]

    @property
    def improvements(self) -> List[Finding]:
        return [f for f in self.findings if f.kind == "improvement"]

    @property
    def stale(self) -> List[Finding]:
        return [f for f in self.findings if f.kind == "stale"]

    def overall_status(self) -> str:
        if self.regressions:
            return "REGRESSION"
        if self.stale:
            return "STALE"
        return "PASS"

    def exit_code(self) -> int:
        if self.regressions:
            return 1
        if self.stale:
            return 4
        return 0


def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) -> NightlyResult:
    """Diff current run vs frozen baseline.

    * Segmenter changed ? STALE (numbers not comparable; re-freeze).
    * Corpus (video set) changed ? STALE + report added/removed; still diff the INTERSECTION.
    * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR = regression;
      quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
    """
```

```
    res = NightlyResult()
    if baseline.get("segmenter") != current.get("segmenter"):
        res.findings.append(Finding("", "segmenter", "stale",
                                    detail=f"{baseline.get('segmenter')} ? {current.get('segmenter')}"))
        return res

    b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
    res.added = sorted(set(c_pv) - set(b_pv))
    res.removed = sorted(set(b_pv) - set(c_pv))
    if res.added or res.removed:
        res.findings.append(Finding("", "corpus", "stale",
                                    detail=f"added={res.added} removed={res.removed}"))

    for v in sorted(set(b_pv) & set(c_pv)):
        b, c = b_pv[v], c_pv[v]
        # 1) pipeline must still succeed
        if not c.get("ok", False) or c.get("reel_count") is None:
            res.findings.append(Finding(v, "error", "regression",
                                        detail=str(c.get("error") or "no reel_count produced")))
            continue
        # 2) reel count ? EXACT
        bc, cc = b.get("reel_count"), c.get("reel_count")
        if bc is not None and cc is not None and cc != bc:
            res.findings.append(Finding(v, "reel_count", "regression", float(bc), float(cc)))
        # 3) quality metrics ? tolerance (only if both sides have them)
        bq, cq = b.get("quality") or {}, c.get("quality") or {}
        for m in QUALITY_HIGHER:
            bv, cv = bq.get(m), cq.get(m)
            if bv is None or cv is None:
                continue
            if cv < bv - tols.quality_tol:
                res.findings.append(Finding(v, m, "regression", bv, cv))
            elif cv > bv + tols.quality_tol:
                res.findings.append(Finding(v, m, "improvement", bv, cv))
    return res


# --------------------------------------------------------------------------- #
# Report formatting (pure).
# --------------------------------------------------------------------------- #
def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
    if not qd or qd.get(key) is None:
        return "?"
    return f"{qd[key]:.3f}"


def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
                  result: NightlyResult, date: str, head: str) -> str:
    status = result.overall_status()
    badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
             "STALE": "? STALE (re-freeze the baseline)"}[status]
    cost = current.get("cost", {})
    L: List[str] = [
        f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
        "",
        f"**Status: {badge}**  ·  exit code {result.exit_code()}  ·  segmenter `{current.get('segmenter')}`",
        "",
        f"- HEAD `{head}`  ·  baseline `{baseline.get('git_commit', '?')}` (frozen {baseline.get('generated_at', '?')})",
        f"- **Run cost: ${cost.get('usd', 0):.4f} (?{cost.get('inr', 0):.2f})**  ·  "
        f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
```

```
0)}/hr",
        "",
    ]
    if result.regressions:
        L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions] + [""]
    if result.stale:
        L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in result.stale]
        L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after confirming the change is intended).", ""]

    # Per-video table.
    b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
    L += ["## Per-video", "",
          "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
          "|---|---|---|---|---|"]
    for v in sorted(set(b_pv) | set(c_pv)):
        b, c = b_pv.get(v, {}), c_pv.get(v, {})
        bc = b.get("reel_count")
        cc = c.get("reel_count")
        rc = f"{'?' if bc is None else bc}?{'ERROR' if not c.get('ok', True) else ('?' if cc is None else cc)}"
        bq, cq = b.get("quality"), c.get("quality")
        tf = f"{_q(bq, 'tol_f1')}?{_q(cq, 'tol_f1')}"
        f5 = f"{_q(bq, 'f1@0.5')}?{_q(cq, 'f1@0.5')}"
        vstatus = "?" if any(f.video == v and f.kind == "regression" for f in result.findings) else "?"
        L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
    L.append("")

    if result.improvements:
        L += ["_Improvements over baseline (re-freeze to ratchet up):_"] \
            + [f"- {f.message()}" for f in result.improvements] + [""]
    L += ["---",
          "_Full configured+checkpointed pipeline on the real model (GCP GPU VM). Tripwire only ? does "
          "not change the promotion gate. Complements PR #202's cached-trajectory CPU re-score._"]
    return "\n".join(L)


# ------------------------------------------------------------------------ #
# IO shell ? running the configured pipeline per video (needs backend + GPU + video).
# ------------------------------------------------------------------------ #
def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
    """Load ground-truth rally intervals from a golden-features JSON (``gts`` key) or a rallies CSV
    (``start,end`` columns / pairs). Returns None when no labels are configured."""
    if not labels_ref:
        return None
    from backend.storage import fetch
    local = fetch(labels_ref)
    if local.endswith(".json"):
        with open(local, encoding="utf-8") as fh:
            data = json.load(fh)
        return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
    # CSV: header start,end (or first two numeric columns)
    import csv
    out: List[Tuple[float, float]] = []
    with open(local, encoding="utf-8") as fh:
        for row in csv.reader(fh):
            try:
                out.append((float(row[0]), float(row[1])))
            except (ValueError, IndexError):
```

```
                continue
    return out


def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
    """``(start, end)`` pairs from segmenter result dicts, tolerant of the two key conventions in the
    codebase (``start_time``/``end_time`` ? motion/DB shape ? or ``start``/``end``). Used only for the
    optional quality metrics; the reel COUNT is ``len(segs)`` regardless, so a key mismatch degrades
    quality scoring gracefully (skipped) without ever miscounting reels."""
    out: List[Tuple[float, float]] = []
    for s in segs:
        start = s.get("start_time", s.get("start"))
        end = s.get("end_time", s.get("end"))
        if start is not None and end is not None:
            out.append((float(start), float(end)))
    return out


def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
                  rerun_on_mismatch: bool = True,
                  baseline_count: Optional[int] = None) -> Dict[str, Any]:
    """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy backend imports.

    The re-run-once guard (the one known non-determinism ? a transient ``add_play_segment``?None drop,
    database.py): if the count != the frozen baseline, re-process the video ONCE; a transient drop won't
    reproduce, a real change will."""
    import random
    import time

    from backend.eval import rally_seg_eval
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters import segmenter_registry
    from backend.results import Failure
    from backend.storage import fetch
    from backend.utils.hashing import compute_video_id

    name = video["name"]
    try:
        local = fetch(video["uri"])
    except Exception as e:  # noqa: BLE001 ? surface, never hide
        return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed: {e}",
                "quality": None, "wall_seconds": 0.0}

    gts = _load_gts(video.get("labels_uri"))
    seg_cls = segmenter_registry.get(segmenter)
    if not seg_cls:
        return {"name": name, "reel_count": None, "ok": False,
                "error": f"segmenter '{segmenter}' not registered", "quality": None, "wall_seconds": 0.0}

    def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float, float]]], float, Optional[str]]:
        random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields are reproducible too
        db = Database(db_path)
        video_id = compute_video_id(local)
        t0 = time.monotonic()
        result = seg_cls(db, config).process_video(video_id, local)
```

```
        wall = time.monotonic() - t0
        if isinstance(result, Failure):
            return None, None, wall, getattr(result, "message", "segmenter failed")
        segs = result.value if hasattr(result, "value") else result
        intervals = _segments_to_intervals(segs)
        return len(segs), intervals, wall, None

    import tempfile
    total_wall = 0.0
    with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
        count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
        total_wall += wall
        if err is not None:
            return {"name": name, "reel_count": None, "ok": False, "error": err,
                    "quality": None, "wall_seconds": round(total_wall, 2)}
        # re-run-once guard for the transient DB-drop flake
        if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
            count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
            total_wall += wall2
            if err2 is None and count2 is not None:
                count, intervals = count2, intervals2  # the reproduced (or corrected) value

        quality = None
        if gts and intervals is not None:
            row = rally_seg_eval.score_intervals(intervals, gts)
            quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25", "tol_precision", "tol_recall")
                       if k in row}

    return {"name": name, "reel_count": count, "ok": True, "error": None,
            "quality": quality, "wall_seconds": round(total_wall, 2)}


def load_manifest(path: str) -> Dict[str, Any]:
    with open(path, encoding="utf-8") as fh:
        return json.load(fh)


def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
               rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
    """Run every manifest video through the configured pipeline. Returns (run_results, total_wall_s)."""
    from backend.config import load_config

    config = load_config()
    # apply manifest config overrides onto the typed config (dotted keys)
    for dotted, val in (manifest.get("config_overrides") or {}).items():
        _set_dotted(config, dotted, val)
    segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None), "default_segmenter", "motion")
    b_pv = (baseline or {}).get("per_video", {})
    results: List[Dict[str, Any]] = []
    total_wall = 0.0
    for v in manifest.get("videos", []):
        base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
        r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
        total_wall += float(r.get("wall_seconds") or 0.0)
        results.append(r)
    return results, total_wall


def _set_dotted(config: Any, dotted: str, value: Any) -> None:
    """Best-effort dotted-path set onto a pydantic config (e.g. 'indexing.motion_threshold')."""
```

```
    parts = dotted.split(".")
    obj = config
    for p in parts[:-1]:
        obj = getattr(obj, p, None)
        if obj is None:
            return
    try:
        setattr(obj, parts[-1], value)
    except Exception:  # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't crash the run
        pass


def _git_head() -> str:
    import subprocess
    try:
        out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
                             capture_output=True, text=True, timeout=10)
        return out.stdout.strip() or "?"
    except Exception:  # noqa: BLE001
        return "?"


def load_baseline(path: str) -> Optional[Dict[str, Any]]:
    if not os.path.exists(path):
        return None
    with open(path, encoding="utf-8") as fh:
        return json.load(fh)


def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
    out = dict(snapshot)
    out["generated_at"] = _dt.date.today().isoformat()
    out["git_commit"] = _git_head()
    out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python scripts/nightly_reelcount.py "
                   "--update-baseline` after an INTENDED engine change or when adding a video. Tied to the "
                   "configured+checkpointed engine + the manifest corpus.")
    out.pop("cost", None)  # cost is per-run, not part of the frozen baseline
    os.makedirs(os.path.dirname(path), exist_ok=True)
    tmp = path + ".tmp"
    with open(tmp, "w", encoding="utf-8") as fh:
        json.dump(out, fh, indent=2, sort_keys=True)
        fh.write("\n")
    os.replace(tmp, path)


def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count + quality + cost.")
    p.add_argument("--manifest", default=DEFAULT_MANIFEST)
    p.add_argument("--baseline", default=DEFAULT_BASELINE)
    p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
    p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
    p.add_argument("--vm-uptime-seconds", type=float, default=None,
                   help="authoritative VM uptime (boot?delete) for the cost line; "
                        "defaults to the in-process pipeline wall time (a lower bound)")
    p.add_argument("--no-rerun-guard", action="store_true",
                   help="disable the re-run-once-on-mismatch guard (the DB-drop flake mitigation)")
    p.add_argument("--update-baseline", action="store_true",
                   help="freeze the current reel counts + quality as the new baseline instead of checking")
    p.add_argument("--email", action="store_true", help="email the report (see
```

```
scripts/nightly_email.py)")
    p.add_argument("--email-to", default=None, help="override the report recipient")
    p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD; for tests)")
    p.add_argument("--no-report", action="store_true", help="skip writing the dated report file")
    return p


def main(argv: Optional[List[str]] = None) -> int:
    import logging
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
    log = logging.getLogger("nightly_reelcount")
    args = build_parser().parse_args(argv)
    tols = Tolerances(quality_tol=args.quality_tol)
    date = args.date or _dt.date.today().isoformat()

    if not os.path.exists(args.manifest):
        log.error("manifest not found: %s", args.manifest)
        return 2
    manifest = load_manifest(args.manifest)
    baseline = load_baseline(args.baseline)

    try:
        run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not args.no_rerun_guard)
    except Exception as e:  # noqa: BLE001 ? operational failure (deps/config) ? exit 2, never a false "pass"
        log.error("run failed: %s", e)
        return 2
    if not run_results:
        log.error("no videos in manifest %s", args.manifest)
        return 2

    billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else total_wall
    cost = cost_report(
        billed_seconds=billed,
        machine_usd_per_hr=float(manifest.get("machine_usd_per_hr", DEFAULT_MACHINE_USD_PER_HR)),
        fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
        gpu_seconds=total_wall,
    )
    segmenter = manifest.get("segmenter", "motion")
    snapshot = summarize_results(run_results, segmenter, cost)
    snapshot["git_commit"] = _git_head()
    snapshot["generated_at"] = date

    if args.update_baseline:
        save_baseline(args.baseline, snapshot)
        log.info("baseline frozen ? %s", args.baseline)
        for r in run_results:
            log.info("  %s: reels=%s ok=%s", r["name"], r.get("reel_count"), r.get("ok"))
        return 0

    if baseline is None:
        log.error("no baseline at %s ? freeze one with --update-baseline (on the configured engine).", args.baseline)
        return 3

    result = compare(baseline, snapshot, tols)
    report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])

    if not args.no_report:
```

```
        os.makedirs(args.report_dir, exist_ok=True)
        md_path = os.path.join(args.report_dir, f"{date}.md")
        with open(md_path, "w", encoding="utf-8") as fh:
            fh.write(report + "\n")
        with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8") as fh:
            json.dump({"date": date, "status": result.overall_status(), "exit_code": result.exit_code(),
                       "snapshot": snapshot, "regressions": [f.message() for f in result.regressions]},
                      fh, indent=2, sort_keys=True)
        log.info("report ? %s", md_path)

    print(report)

    if args.email:
        try:
            from scripts.nightly_email import send_report
            subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count engine regression {date}"
            sent = send_report(subject, report, to=args.email_to)
            log.info("email %s", "sent" if sent else "skipped (no creds ? report still written)")
        except Exception as e:  # noqa: BLE001 ? a broken mailer must NOT mask the regression verdict
            log.error("email failed (report still written): %s", e)

    code = result.exit_code()
    if code == 1:
        log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions), args.report_dir)
    elif code == 4:
        log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-baseline")
    else:
        log.info("nightly reel-count check PASSED  (cost $%.4f / ?%.2f)", cost["usd"], cost["inr"])
    return code


if __name__ == "__main__":
    raise SystemExit(main())

```

4. user (2026-06-22T09:48:22.068Z)

```
"""Email the nightly regression report ? stdlib SMTP, creds from env or GCP Secret Manager.

Used by ``scripts/nightly_reelcount.py --email`` (and reusable by any nightly). Deliberately
dependency-light (stdlib ``smtplib`` + ``email.message``) and **graceful**: if no SMTP password can
be resolved it logs + returns ``False`` (the report is still written/printed ? a broken mailer must
never mask a regression).

Credential resolution (first hit wins), so it works on a GCP VM, locally, or in tests:
  ? Recipient:  ``--email-to`` arg  ?  ``NIGHTLY_EMAIL_TO``  ?  ``avi.dullu@gmail.com``
  ? SMTP host:  ``NIGHTLY_SMTP_HOST``  (default ``smtp.gmail.com``)   · port ``NIGHTLY_SMTP_PORT`` (587, STARTTLS)
  ? SMTP user:  ``NIGHTLY_SMTP_USER``  (default = recipient)
  ? SMTP pass:  ``NIGHTLY_SMTP_PASS``  ?  Secret Manager ``NIGHTLY_SMTP_SECRET``
                (``projects/<p>/secrets/<name>/versions/latest``; via the google-cloud-secret-manager
```

```
                 lib if installed, else the ``gcloud`` CLI). A Gmail **app password** is the simplest.

Set the secret once (owner):  echo -n '<app-password>' | gcloud secrets create nightly-smtp-pass --data-file=-
and on the VM:  export NIGHTLY_SMTP_SECRET=projects/<proj>/secrets/nightly-smtp-pass/versions/latest
                export NIGHTLY_SMTP_USER=<your-gmail> NIGHTLY_EMAIL_TO=avi.dullu@gmail.com
"""
from __future__ import annotations

import logging
import os
from email.message import EmailMessage
from typing import Optional

log = logging.getLogger("nightly_email")

DEFAULT_TO = "avi.dullu@gmail.com"
DEFAULT_HOST = "smtp.gmail.com"
DEFAULT_PORT = 587


def build_message(subject: str, body_md: str, from_addr: str, to_addr: str) -> EmailMessage:
    """A text/plain email carrying the markdown report (renders fine in any client). Pure ? testable."""
    msg = EmailMessage()
    msg["Subject"] = subject
    msg["From"] = from_addr
    msg["To"] = to_addr
    msg.set_content(body_md)
    return msg


def resolve_secret(env: Optional[dict] = None) -> Optional[str]:
    """Resolve the SMTP password from env or GCP Secret Manager. Returns None if unresolvable."""
    env = env if env is not None else os.environ
    direct = env.get("NIGHTLY_SMTP_PASS")
    if direct:
        return direct
    secret_ref = env.get("NIGHTLY_SMTP_SECRET")
    if not secret_ref:
        return None
    # Prefer the client lib; fall back to the gcloud CLI (no extra dependency required).
    try:
        from google.cloud import secretmanager  # type: ignore
        client = secretmanager.SecretManagerServiceClient()
        return client.access_secret_version(name=secret_ref).payload.data.decode("utf-8")
    except Exception:  # noqa: BLE001 ? lib absent or auth issue ? try the CLI
        pass
    try:
        import subprocess
        name = secret_ref.split("/secrets/", 1)[1].split("/", 1)[0] if "/secrets/" in secret_ref else secret_ref
        out = subprocess.run(["gcloud", "secrets", "versions", "access", "latest", "--secret", name],
                             capture_output=True, text=True, timeout=30)
        return out.stdout.strip() if out.returncode == 0 and out.stdout.strip() else None
    except Exception as e:  # noqa: BLE001
        log.warning("secret-manager resolution failed: %s", e)
```

```
        return None


def send_report(subject: str, body_md: str, to: Optional[str] = None,
                env: Optional[dict] = None) -> bool:
    """Send the report. Returns True if sent, False if skipped (no creds) ? never raises on missing creds."""
    env = env if env is not None else os.environ
    to_addr = to or env.get("NIGHTLY_EMAIL_TO") or DEFAULT_TO
    host = env.get("NIGHTLY_SMTP_HOST", DEFAULT_HOST)
    port = int(env.get("NIGHTLY_SMTP_PORT", DEFAULT_PORT))
    user = env.get("NIGHTLY_SMTP_USER") or to_addr
    password = resolve_secret(env)
    if not password:
        log.warning("no SMTP password (set NIGHTLY_SMTP_PASS or NIGHTLY_SMTP_SECRET) ? email skipped.")
        return False

    import smtplib
    msg = build_message(subject, body_md, from_addr=user, to_addr=to_addr)
    with smtplib.SMTP(host, port, timeout=60) as s:
        s.starttls()
        s.login(user, password)
        s.send_message(msg)
    log.info("emailed report to %s", to_addr)
    return True

```

5. user (2026-06-22T09:48:22.697Z)

EISDIR: illegal operation on a directory, read 'C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\nightly-e2e\deploy\gcp\nightly_regression'

6. assistant (2026-06-22T09:48:24.800Z)

Let me read the deployment scripts separately:

7. user (2026-06-22T09:48:26.139Z)

deploy\gcp\nightly_regression\run_on_vm.sh
deploy\gcp\nightly_regression\launch.sh
deploy\gcp\nightly_regression\Dockerfile.launcher
deploy\gcp\nightly_regression\register_scheduler.sh
deploy\gcp\nightly_regression\README.md

8. assistant (2026-06-22T09:48:28.483Z)

Now let me read these deployment files and the manifest:

9. user (2026-06-22T09:48:29.229Z)

```
#!/usr/bin/env bash
# Startup-script for the NIGHTLY engine-regression VM. Runs UNATTENDED on boot, then DELETES the VM
# (cost rule: a running GPU VM is the only real cost). Survives SSH drops (it's a startup-script, not
# an interactive session). Reads its config from instance metadata (set by launch.sh).
#
# Flow:  Ops Agent  ?  pull repo @ HEAD from GCS  ?  build venv + WASB env  ?  stage weights  ?
#        run scripts/nightly_reelcount.py on the gs:// golden clips (emails the report) ?
#        upload report to GCS  ?  SELF-DELETE (even on failure, via the EXIT trap).
#
```

```
# Metadata keys (launch.sh sets these): repo-tgz-uri, git-sha, gcs-bucket, report-prefix,
#   weights-uri, email (true/false). NOT verified in CI ? dry-run via launch.sh on a real VM first.
set -uo pipefail
exec > >(tee -a /var/log/nightly-regression.log) 2>&1
echo "===== nightly-regression startup $(date -u +%FT%TZ) ====="
BOOT_EPOCH=$(date +%s)

md() { curl -s -H "Metadata-Flavor: Google" "http://metadata.google.internal/computeMetadata/v1/$1"; }
NAME=$(md instance/name)
ZONE=$(md instance/zone | awk -F/ '{print $NF}')
REPO_TGZ=$(md instance/attributes/repo-tgz-uri || true)
GIT_SHA=$(md instance/attributes/git-sha || echo unknown)
BUCKET=$(md instance/attributes/gcs-bucket || true)
REPORT_PREFIX=$(md instance/attributes/report-prefix || echo nightly/reports)
WEIGHTS_URI=$(md instance/attributes/weights-uri || true)
DO_EMAIL=$(md instance/attributes/email || echo true)
SMTP_SECRET=$(md instance/attributes/smtp-secret || true)
SMTP_USER=$(md instance/attributes/smtp-user || true)
EMAIL_TO=$(md instance/attributes/email-to || echo avi.dullu@gmail.com)
DATE=$(date -u +%F)

# ALWAYS delete the VM on exit (success OR failure) ? never leave a billing GPU VM behind.
cleanup() {
  echo "===== self-delete $NAME ($ZONE) ====="
  gcloud compute instances delete "$NAME" --zone="$ZONE" --quiet || \
    echo "WARN: self-delete failed ? DELETE MANUALLY: gcloud compute instances delete $NAME --zone=$ZONE -q"
}
trap cleanup EXIT

fail_email() {  # best-effort failure alert (the report email won't have fired on an early crash)
  echo "RUN FAILED ? attempting a failure alert"
  if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
    cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
    NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER" NIGHTLY_EMAIL_TO="$EMAIL_TO" \
    python - <<PY || true
from scripts.nightly_email import send_report
send_report("[KhelSutra nightly] ERROR ? engine regression VM run failed ($DATE)",
            "The nightly engine-regression run failed on the VM before producing a report.\n"
            "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial log.\nGIT_SHA=$GIT_SHA", to="$EMAIL_TO")
PY
  fi
}
trap 'fail_email' ERR

# 1) Observability (Ops Agent) ? same as create_gpu_vm.sh bakes in.
curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && \
  sudo bash add-google-cloud-ops-agent-repo.sh --also-install || echo "WARN: ops-agent install failed (non-fatal)"

# 2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses the worker SA).
mkdir -p ~/rally && cd ~/rally
gcloud storage cp "$REPO_TGZ" /tmp/rally.tgz
tar -xzf /tmp/rally.tgz -C ~/rally
echo "$GIT_SHA" > ~/rally/GIT_SHA

```

```
# 3) Build the env + native WASB (provider-agnostic DO scripts, reused on GCP per deploy/gcp/README §4).
sudo bash deploy/digitalocean/mount_scratch.sh 2>/dev/null || true
export TMPDIR=/scratch 2>/dev/null || true
./deploy/digitalocean/bootstrap.sh --gpu
. .venv/bin/activate
pip install -e '.[storage-gcs]'
bash deploy/digitalocean/gpu_setup.sh || echo "WARN: gpu_setup (WASB env) returned nonzero ? continuing"

# 4) Stage the WASB checkpoint (gpu_setup does NOT fetch weights ? README §4).
if [ -n "$WEIGHTS_URI" ]; then
  mkdir -p ~/models/WASB-SBDT/pretrained_weights
  gcloud storage cp "$WEIGHTS_URI" ~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
  export WASB_WEIGHTS=~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
fi

# 5) Run the regression on the gs:// golden clips in the committed manifest. Cost = VM uptime so far.
UPTIME=$(( $(date +%s) - BOOT_EPOCH ))
EMAIL_FLAG=""; [ "$DO_EMAIL" = "true" ] && EMAIL_FLAG="--email --email-to $EMAIL_TO"
set +e
NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER" NIGHTLY_EMAIL_TO="$EMAIL_TO" \
  python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
RUN_RC=$?
set -e
echo "runner exit code: $RUN_RC  (0 pass / 1 regression / 4 stale)"

# 6) Upload the dated report to GCS for the record (alongside the email).
if [ -n "$BUCKET" ] && [ -d output/nightly_reelcount ]; then
  gcloud storage cp -r "output/nightly_reelcount/$DATE."* "gs://$BUCKET/$REPORT_PREFIX/$DATE/" || \
    gcloud storage cp -r output/nightly_reelcount "gs://$BUCKET/$REPORT_PREFIX/$DATE/" || true
fi

echo "===== nightly-regression done rc=$RUN_RC $(date -u +%FT%TZ) ====="
trap - ERR   # disable the failure-email trap; the EXIT (self-delete) trap stays
# the EXIT trap deletes the VM

```

10. user (2026-06-22T09:48:29.893Z)

```
#!/usr/bin/env bash
# Launch ONE nightly engine-regression run on an ephemeral GCP GPU VM, then return. The VM runs
# run_on_vm.sh as its startup-script (regression ? email ? SELF-DELETE), so this just stages the repo
# and kicks the VM off. Driver-agnostic: call it from Cloud Scheduler?Cloud Run job, a cron on any
# always-on box, the owner's machine, or a Claude routine. Needs gcloud auth + the repo checkout.
#
#   bash deploy/gcp/nightly_regression/launch.sh                 # L4, stages HEAD, kicks the VM
#   RALLY_GPU=t4 RALLY_REF=$(git rev-parse HEAD) bash deploy/gcp/nightly_regression/launch.sh
#
# ? Run this MANUALLY once as a dry-run (watch the serial log) before wiring a scheduler ? the VM-side
# flow is NOT covered by CI. Tail: gcloud compute instances get-serial-port-output <name> --zone=<z>.
set -uo pipefail
```

```
PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
BUCKET="${RALLY_BUCKET:-khelsutra-rally-corpus}"
SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
GPU="${RALLY_GPU:-l4}"
NAME="${RALLY_VM_NAME:-rally-nightly-$(date -u +%Y%m%d)}"
REF="${RALLY_REF:-HEAD}"
WEIGHTS_URI="${RALLY_WEIGHTS_URI:-gs://$BUCKET/weights/wasb_badminton_best.pth.tar}"
REPORT_PREFIX="${RALLY_REPORT_PREFIX:-nightly/reports}"
DO_EMAIL="${RALLY_EMAIL:-true}"
SMTP_SECRET="${NIGHTLY_SMTP_SECRET:-projects/$PROJECT/secrets/nightly-smtp-pass/versions/latest}"
SMTP_USER="${NIGHTLY_SMTP_USER:-}"
EMAIL_TO="${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
STARTUP="$HERE/run_on_vm.sh"
export CLOUDSDK_CORE_DISABLE_PROMPTS=1

ZONES="${RALLY_GCP_ZONES:-asia-south1-a asia-south1-b asia-south1-c asia-southeast1-a asia-southeast1-b asia-southeast1-c}"
if [ "$GPU" = "l4" ]; then MACHINE="g2-standard-4"; ACCEL=(); else MACHINE="n1-standard-8"; ACCEL=(--accelerator=type=nvidia-tesla-t4,count=1); fi

# 1) Stage the repo @ REF to GCS (no GitHub auth on the VM; the worker SA can read the bucket).
SHA="$(git rev-parse "$REF")"
TGZ_URI="gs://$BUCKET/nightly/repo/${SHA}.tgz"
echo "== staging repo $SHA -> $TGZ_URI =="
TMP_TGZ="$(mktemp --suffix=.tgz)"
git archive --format=tar.gz -o "$TMP_TGZ" "$REF"
gcloud storage cp "$TMP_TGZ" "$TGZ_URI" --project="$PROJECT"
rm -f "$TMP_TGZ"

# 2) Zone-sweep create the VM (mirrors create_gpu_vm.sh) with run_on_vm.sh + the config metadata.
ERR="$(mktemp)"
for Z in $ZONES; do
  echo "== creating $NAME ($GPU) in $Z =="
  if gcloud compute instances create "$NAME" --project="$PROJECT" --zone="$Z" \
       --machine-type="$MACHINE" "${ACCEL[@]}" \
       --maintenance-policy=TERMINATE --provisioning-model=STANDARD \
       --image-family=common-cu129-ubuntu-2204-nvidia-580 --image-project=deeplearning-platform-release \
       --boot-disk-size=120GB --boot-disk-type=pd-balanced \
       --service-account="$SA" --scopes=cloud-platform \
       --metadata-from-file=startup-script="$STARTUP" \
       --metadata=repo-tgz-uri="$TGZ_URI",git-sha="$SHA",gcs-bucket="$BUCKET",report-prefix="$REPORT_PREFIX",weights-uri="$WEIGHTS_URI",email="$DO_EMAIL",smtp-secret="$SMTP_SECRET",smtp-user="$SMTP_USER",email-to="$EMAIL_TO" \
       2>"$ERR"; then
    echo "? launched $NAME in $Z ? it will run the regression, email $EMAIL_TO, and self-delete."
    echo "   serial log: gcloud compute instances get-serial-port-output $NAME --zone=$Z --project=$PROJECT"
    echo "   if it doesn't self-delete: gcloud compute instances delete $NAME --zone=$Z -q"
    rm -f "$ERR"; exit 0
  fi
  if grep -qiE "STOCKOUT|does not have enough" "$ERR"; then echo "  (no $GPU capacity in $Z ? next zone)"; continue; fi
  echo "  create failed:"; tail -4 "$ERR"; rm -f "$ERR"; exit 1
done
echo "? no $GPU capacity in any swept zone ($ZONES). Retry later or RALLY_GPU=t4."
rm -f "$ERR"; exit 1
```

11. user (2026-06-22T09:48:30.567Z)

```
#!/usr/bin/env bash
# Wire the nightly engine-regression as a CLOUD-NATIVE, scale-to-zero job:
#   Cloud Scheduler (cron)  ?  Cloud Run JOB (the launcher image)  ?  launch.sh
#     ?  ephemeral GPU VM (regression + email + SELF-DELETE).
# The only always-on cost is $0 (Cloud Run jobs + Scheduler are scale-to-zero; the GPU VM is ephemeral).
#
# ? OWNER-RUN, ONCE, after a successful manual `bash deploy/gcp/nightly_regression/launch.sh` dry-run.
# NOT verified in CI (no GCP/GPU here). Prereqs: GPU quota (README §2), the golden clips + weights in
# gs://<bucket>, and the SMTP secret (see README "Enable").
#
#   bash deploy/gcp/nightly_regression/register_scheduler.sh           # build + deploy + schedule (03:00 IST)
#   RALLY_CRON='0 3 * * *' RALLY_TZ='Asia/Kolkata' bash .../register_scheduler.sh
set -euo pipefail

PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
REGION="${RALLY_RUN_REGION:-asia-south1}"
JOB="${RALLY_RUN_JOB:-nightly-regression-launcher}"
SCHED="${RALLY_SCHED_NAME:-nightly-regression}"
CRON="${RALLY_CRON:-0 3 * * *}"          # 03:00 daily
TZ="${RALLY_TZ:-Asia/Kolkata}"
SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
IMAGE="${RALLY_LAUNCHER_IMAGE:-${REGION}-docker.pkg.dev/${PROJECT}/rally/nightly-launcher:latest}"
REPO_ROOT="$(git rev-parse --show-toplevel)"
export CLOUDSDK_CORE_DISABLE_PROMPTS=1

echo "== enable APIs (run, scheduler, build, artifact registry) =="
gcloud services enable run.googleapis.com cloudscheduler.googleapis.com \
  cloudbuild.googleapis.com artifactregistry.googleapis.com --project="$PROJECT"

echo "== ensure the Artifact Registry repo 'rally' exists =="
gcloud artifacts repositories describe rally --location="$REGION" --project="$PROJECT" >/dev/null 2>&1 || \
  gcloud artifacts repositories create rally --repository-format=docker --location="$REGION" --project="$PROJECT"

echo "== build + push the launcher image from the repo root =="
( cd "$REPO_ROOT" && gcloud builds submit --project="$PROJECT" --tag "$IMAGE" \
    --gcs-source-staging-dir="gs://${RALLY_BUILD_BUCKET:-khelsutra-rally-corpus}/cloudbuild" \
    -f deploy/gcp/nightly_regression/Dockerfile.launcher . )

echo "== create/update the Cloud Run JOB (launcher needs compute-admin to create the GPU VM) =="
gcloud run jobs deploy "$JOB" --project="$PROJECT" --region="$REGION" \
  --image="$IMAGE" --service-account="$SA" --max-retries=0 --task-timeout=900s \
  --set-env-vars="RALLY_GPU=${RALLY_GPU:-l4},RALLY_EMAIL=true,NIGHTLY_EMAIL_TO=${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"

echo "== schedule it (Cloud Scheduler ? Run job admin API) =="
RUN_URI="https://${REGION}-run.googleapis.com/apis/run.googleapis.com/v1/namespaces/${PROJECT}/jobs/${JOB}:run"
gcloud scheduler jobs describe "$SCHED" --location="$REGION" --project="$PROJECT" >/dev/null 2>&1 && \
  gcloud scheduler jobs delete "$SCHED" --location="$REGION" --project="$PROJECT" --quiet || true
gcloud scheduler jobs create http "$SCHED" --project="$PROJECT" --location="$REGION" \
  --schedule="$CRON" --time-zone="$TZ" \
  --uri="$RUN_URI" --http-method=POST \
```

```
    --oauth-service-account-email="$SA"

cat <<EOF

? Scheduled '$SCHED' ($CRON $TZ) ? Cloud Run job '$JOB' ? launch.sh ? ephemeral GPU VM.
   IAM the launcher SA needs (grant once if missing):
     roles/compute.instanceAdmin.v1  (create/delete the GPU VM)
     roles/iam.serviceAccountUser    (attach $SA to the VM)
     roles/run.invoker + roles/cloudscheduler.jobRunner (Scheduler ? Run)
   Trigger now to test:  gcloud run jobs execute $JOB --region=$REGION --project=$PROJECT
   Remove:               gcloud scheduler jobs delete $SCHED --location=$REGION; gcloud run jobs delete $JOB --region=$REGION
EOF

```

12. user (2026-06-22T09:48:31.557Z)

```
# Tiny launcher image for the nightly engine-regression Cloud Run JOB. It is NOT the GPU workload ?
# it only stages the repo + creates the ephemeral GPU VM (which self-deletes). Scale-to-zero: the job
# runs for a few seconds nightly, then exits. Base = google/cloud-sdk (gcloud + git preinstalled).
FROM gcr.io/google.com/cloudsdktool/cloud-sdk:slim
WORKDIR /app
# The repo is the build context (run `gcloud builds submit` from the repo root, see register_scheduler.sh).
COPY . /app
ENTRYPOINT ["bash", "deploy/gcp/nightly_regression/launch.sh"]

```

13. user (2026-06-22T09:48:32.366Z)

```
{
  "_doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row + re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths. labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present => quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your actual short (~5-6 min) OWNED, minors-free clips before enabling.",
  "segmenter": "wasb_hybrid",
  "detector_impl": "native",
  "config_overrides": {
    "indexing.default_segmenter": "wasb_hybrid",
    "indexing.detector_impl": "native"
  },
  "machine_usd_per_hr": 0.70,
  "fx_inr_per_usd": 83.0,
  "videos": [
    {
      "name": "GX030094",
      "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX030094.mp4",
      "labels_uri": "gs://khelsutra-rally-corpus/nightly/labels/GX030094_golden_features.json"
    },
    {
      "name": "Badminton_BXH_2",
      "uri": "gs://khelsutra-rally-corpus/nightly/videos/Badminton_BXH_2.mp4",
      "labels_uri": "gs://khelsutra-rally-corpus/nightly/labels/Badminton_BXH_2_golden_features.json"
    },
    {
```

```
          "name": "GX010128",
          "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX010128.mp4",
          "labels_uri": "gs://khelsutra-rally-corpus/nightly/labels/GX010128_golden_features.json"
        }
      ]
    }

```


14. assistant (2026-06-22T09:48:34.939Z)

Now let me look closely at the finding location and examine the Result/Success contract. Let me search for relevant type definitions and usages:

15. user (2026-06-22T09:48:36.556Z)

```
        def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float, float]]], float, Optional[str]]:
            random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields are reproducible too
            db = Database(db_path)
            video_id = compute_video_id(local)
            t0 = time.monotonic()
            result = seg_cls(db, config).process_video(video_id, local)
            wall = time.monotonic() - t0
            if isinstance(result, Failure):
                return None, None, wall, getattr(result, "message", "segmenter failed")
            segs = result.value if hasattr(result, "value") else result
            intervals = _segments_to_intervals(segs)
            return len(segs), intervals, wall, None

        import tempfile
        total_wall = 0.0
```

16. user (2026-06-22T09:48:38.149Z)

```
backend\worker\__main__.py:39:from backend.results import Failure
tools\generate_highlights.py:42:from backend.results import Failure
backend\pipeline\segmenters\base.py:4:from backend.results import Result  # new standardized contract (Result[Value] == Success[Value] | Failure)
backend\pipeline\segmenters\gemini.py:14:from backend.results import Failure  # Finding B: an unrunnable config (missing key/unknown sport/provider) ? a loud Failure, not a silent 0-rally "success"
backend\pipeline\segmenters\motion.py:7:from backend.results import Success, Failure  # standardized contract
backend\main.py:57:from backend.results import Failure  # standardized error/result contract
scripts\run_process_and_stitch.py:19:from backend.results import unwrap_or_failure
scripts\nightly_reelcount.py:323:    from backend.results import Failure
scripts\run_phase3_eval.py:37:from backend.results import unwrap_or_failure
tests\test_gemini.py:7:from backend.results import Failure
tests\test_base.py:4:from backend.results import Success, Failure, Result  # test the new contract
tests\test_motion_segmenter.py:27:from backend.results import Success, Failure
tests\test_results.py:5:from backend.results import Failure, Success, unwrap_or_failure
```

17. user (2026-06-22T09:48:39.117Z)

```
backend\results.py:27:class Success(Generic[T]):
backend\results.py:42:class Failure:
```

18. assistant (2026-06-22T09:48:41.071Z)

Now let me examine the results.py file to understand the Result contract:

19. user (2026-06-22T09:48:41.636Z)

```
"""Lightweight Result type for standardizing success vs. failure across the pipeline.

This is the start of the Error/Result Contract standardization (low-regret item #3).

The goal is to make it impossible to confuse "no data / no rallies" with "something actually failed"
(e.g. missing API key, WSL healthcheck failure, provider error, etc.).

We use simple dataclasses so callers can use isinstance checks or structural pattern matching.
This is intentionally small and incremental ? we start by applying it to the segmenter
and provider layers (the biggest sources of silent [] returns today).

The types are generic where it makes sense. Callers can still access .value on Success
or .message on Failure.

This module has no dependencies on the rest of the app.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Generic, TypeVar, List, Optional, Any, Tuple, Union

T = TypeVar("T")


@dataclass
class Success(Generic[T]):
    """Represents a successful outcome with an optional list of non-fatal warnings."""

    value: T
    warnings: List[str] = field(default_factory=list)

    def is_success(self) -> bool:
        return True

    def unwrap(self) -> T:
        """Return the value. Only safe to call if you already know it's a Success."""
        return self.value


@dataclass
class Failure:
    """Represents a failure with a human-readable message and optional details."""

    message: str
    details: Optional[dict[str, Any]] = None
    is_recoverable: bool = False  # e.g. missing API key (recoverable) vs internal crash

    def is_success(self) -> bool:
        return False

    def unwrap(self) -> Any:
        """Raise to make misuse obvious in code that assumes success."""
        raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")


# Convenient alias for use in type hints: Result[list[dict], Failure]
# Result[ValueType] means Success[ValueType] | Failure
# (the Failure "error" type is fixed for this contract)
Result = Union[Success[T], Failure]

```

```
def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
    """Split a segmenter/provider return into ``(value, failure)`` so a caller branches once.

    - ``Failure``            -> ``(None, failure)``
    - ``Success(value=v)``   -> ``(v, None)``
    - a bare value (legacy code that predates this contract and still returns a raw list)
      -> ``(value, None)``

    This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline; callers
    that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no len()``
    on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is heterogeneous
    (Success | Failure | legacy bare value)."""
    if isinstance(result, Failure):
        return None, result
    value = result.value if hasattr(result, "value") else result
    return value, None

```

20. assistant (2026-06-22T09:48:43.572Z)

Perfect! Now let me check the segmenter base class to understand what process_video should return:

21. user (2026-06-22T09:48:44.880Z)

```
from abc import ABC, abstractmethod
from typing import List, Dict, Any, Type, Optional
from backend.infrastructure.database import Database
from backend.results import Result  # new standardized contract (Result[Value] == Success[Value] | Failure)

class BasePlaySegmenter(ABC):
    def __init__(self, db: Database, config: Dict[str, Any]):
        self.db = db
        self.config = config

    @property
    @abstractmethod
    def name(self) -> str:
        """Unique identifier for this segmenter."""
        pass

    @property
    @abstractmethod
    def description(self) -> str:
        """Human-readable explanation of this segmenter."""
        pass

    @abstractmethod
    def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
        """
        Processes a video to detect play segments, populates SQLite database,
        and returns a Result.

        Use isinstance(result, Failure) (or result.is_success()) to distinguish
        "no segments found" from "something went wrong" (missing key, healthcheck
        failure, etc.). This is the start of the standardized error/result contract.
        """
```

```
        pass

class SegmenterRegistry:
    def __init__(self):
        self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}

    def register(self, segmenter_cls: Type[BasePlaySegmenter]):
        """Registers a new segmenter class under its name property."""
        try:
            temp_inst = segmenter_cls(None, {})
            name = temp_inst.name
        except Exception as e:
            raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
        self._segmenters[name] = segmenter_cls

    def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
        return self._segmenters.get(name)

    def list_available(self) -> Dict[str, str]:
        """Returns a dict of registered segmenter names and their descriptions."""
        res = {}
        for name, cls in self._segmenters.items():
            try:
                temp_inst = cls(None, {})
                desc = temp_inst.description
            except Exception:
                desc = "No description available."
            res[name] = desc
        return res

segmenter_registry = SegmenterRegistry()

```

22. assistant (2026-06-22T09:48:46.890Z)

Now let me check the motion.py segmenter to verify it returns Success:

23. user (2026-06-22T09:48:48.284Z)

```
backend\pipeline\segmenters\motion.py-17-    def description(self) -> str:
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py-18-        return
"Traditional CV pixel-motion heuristics tracking frame differences."
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py-19-
backend\pipeline\segmenters\motion.py:20:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py-21-        """
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py-22-        Processes
a video to detect play segments using motion heuristics,
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py-23-        populates
SQLite with detected segments, and indexes detailed events.
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\base.py-21-        pass
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\base.py-22-
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\base.py-23-    @abstractmethod
backend\pipeline\segmenters\base.py:24:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
```

```
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\base.py-25-        """
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\base.py-26-        Processes a
video to detect play segments, populates SQLite database,
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\base.py-27-        and returns
a Result.
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\gemini.py-506-
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\gemini.py-507-        return
segments_data
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\gemini.py-508-
backend\pipeline\segmenters\gemini.py:509:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\gemini.py-510-        """
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\gemini.py-511-
Processes a video to detect play segments using Google Gemini Multimodal Video API,
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\gemini.py-512-
populates SQLite, and indexes detailed events.
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\trajectory_hybrid.py-135-
re-scoring."""
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\trajectory_hybrid.py-136-
return list(range(len(windows)))
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\trajectory_hybrid.py-137-
backend\pipeline\segmenters\trajectory_hybrid.py:138:    def process_video(self, video_id: str,
video_path: str,  # type: ignore[override]
backend\pipeline\segmenters\trajectory_hybrid.py-139-                      player_pool:
Optional[List[str]] = None,
backend\pipeline\segmenters\trajectory_hybrid.py-140-                      max_frames:
Optional[int] = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\trajectory_hybrid.py-141-        # override-ignore: the ABC declares
the Result contract (Success|Failure)
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\yolo_hybrid.py-363-
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\yolo_hybrid.py-364-
return segments_data
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\yolo_hybrid.py-365-
backend\pipeline\segmenters\yolo_hybrid.py:366:    def process_video(self, video_id: str,
video_path: str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\yolo_hybrid.py-367-
self.person.lazy_load_model()
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\yolo_hybrid.py-368-
backend\pipeline\segmenters\yolo_hybrid.py-369-        # Phase 0: resolve sport profile + AI
provider (early-returns [] on any failure).
```

24. assistant (2026-06-22T09:48:50.248Z)

I see a critical mismatch! Let me check the motion.py and trajectory_hybrid implementations more carefully:

25. user (2026-06-22T09:48:51.604Z)

```
import cv2
import time
import random
from typing import List, Dict, Any, Tuple
import logging
from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
from backend.results import Success, Failure  # standardized contract

logger = logging.getLogger(__name__)

class MotionPlaySegmenter(BasePlaySegmenter):
    @property
    def name(self) -> str:
        return "motion"

    @property
    def description(self) -> str:
        return "Traditional CV pixel-motion heuristics tracking frame differences."

    def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
        """
        Processes a video to detect play segments using motion heuristics,
        populates SQLite with detected segments, and indexes detailed events.
        """
        if not player_pool:
            player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]

        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            logger.error(f"Segmenter could not open video: {video_path}")
            return Failure(message=f"Could not open video: {video_path}", is_recoverable=False)

        final_segments = self._detect_motion_segments(cap, max_frames)

        segments_data = self._populate_db_with_events(video_id, final_segments, player_pool)

        return Success(value=segments_data)

    def _detect_motion_segments(self, cap: Any, max_frames: int = None) -> List[Tuple[float, float]]:
        """Run the pixel-motion heuristic over ``cap`` and return the detected
        (start, end) play segments.

        Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
        thresholds it into raw segments, then merges dead-time gaps and filters
        out segments shorter than ``min_segment_duration``. ``cap`` is released
        before returning. This is the detection half of ``process_video`` ? it
        performs no DB writes and no fabrication.
        """
        fps = cap.get(cv2.CAP_PROP_FPS)
        total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
        duration = total_frames / fps if fps > 0 else 0

        # Sub-sample settings (5 FPS for analysis)
        analysis_fps = 5.0
        frame_interval = max(1, int(fps / analysis_fps))

        idx_cfg = self.config.get("indexing", {})
        motion_threshold = idx_cfg.get("motion_threshold", 0.08)
        min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
```

```
        dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)

        prev_gray = None
        motion_signals = []
        timestamps = []

        frame_idx = 0
        processed_count = 0
        t_start = time.time()
        # Emit a progress line roughly every 5% of the video so long runs are
        # observable instead of silent (a full 1080p match is ~tens of minutes).
        progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
        next_progress = progress_step
        logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
                    f"(~{duration/60:.1f} min), analyzing every {frame_interval}th frame")
        while True:
            # Skip decoding for intermediate frames using cap.grab() for high performance
            for _ in range(frame_interval - 1):
                if not cap.grab():
                    break
                frame_idx += 1

            ret, frame = cap.read()
            if not ret:
                break

            # Process frame
            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            # Apply Gaussian Blur to smooth noise
            gray = cv2.GaussianBlur(gray, (21, 21), 0)

            t = frame_idx / fps
            timestamps.append(t)

            if prev_gray is not None:
                # Frame absolute difference (highly optimized motion tracking)
                diff = cv2.absdiff(prev_gray, gray)
                _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)

                # Dilate to fill gaps
                thresh = cv2.dilate(thresh, None, iterations=2)
```

26. user (2026-06-22T09:48:52.520Z)

```
"""Shared engine for trajectory-substrate hybrid segmenters.

tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
now the single home for the whole pipeline shape:

    healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
    -> P3 AI-provider handoff over padded clips -> P4 DB population/linking

A concrete subclass supplies only its identity and runner wiring:
- ``family``        ? config section under ``indexing.*``, output subdir, and
                      the ``<family>-hybrid-<model>`` detector tag in metadata.
- ``display_label`` ? human label for log lines ("WASB", "TrackNet").
- ``name`` / ``description`` properties (the registry contract).
- ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
                      detection_params extras such as weights_path).

This base is deliberately NOT registered; only concrete subclasses register.
"""
```

```python
import os
import time
import logging
import concurrent.futures
from typing import Any, Dict, List, Optional, Tuple

import cv2

from backend.pipeline.segmenters.base import BasePlaySegmenter
from backend.utils.video import get_video_duration, extract_video_chunk
from backend.utils.paths import output_base
from backend.sports import sport_registry
from backend.providers import provider_registry
from backend.pipeline.detectors.base import DetectorRunner

logger = logging.getLogger(__name__)


def _fmt_ts(seconds: float) -> str:
    seconds = max(0, int(seconds))
    h, rem = divmod(seconds, 3600)
    m, s = divmod(rem, 60)
    return f"{h:02d}:{m:02d}:{s:02d}"


class TrajectoryHybridSegmenter(BasePlaySegmenter):
    """Trajectory-detector hybrid: precise candidate rallies + AI semantic boundaries."""

    #: config section under `indexing` (velocity_threshold lives there), the
    #: output subdir for trajectories, and the metadata detector-tag prefix.
    family: str = ""
    #: human-readable label used in log lines.
    display_label: str = ""

    def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        """Return (runner, detector-specific detection_params extras) for the default (WSL) path."""
        raise NotImplementedError

    def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        """Return (runner, extras) for the Linux-native (no-WSL) path. Only families with a
        native runner override this; the base refuses with a clear message (M3.5: WASB only)."""
        raise NotImplementedError(
            f"detector_impl='native' is not supported for the "
            f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB has a "
            f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")

    def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        """Resolve the detector runner, honouring the CPU-stub and Linux-native overrides.

        ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or ``indexing.detector_impl``:
        ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole pipeline
        is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"`` swaps
        in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
```

```
        ``_build_native_runner``. Anything else (``"auto"``) delegates to ``_build_runner``
        (the WSL runner) so the default production path is unchanged
        (``docs/DECOUPLED_COMPUTE/``).
        """
        impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or "auto").lower()
        if impl == "stub":
            from backend.pipeline.detectors.stub_runner import StubDetectorRunner, StubConfig
            logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
                        self.display_label or "trajectory")
            return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)), {"detector_impl": "stub"}
        if impl in ("native", "native_wasb", "wasb_native"):
            logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
                        self.display_label or "trajectory")
            return self._build_native_runner(idx_cfg)
        return self._build_runner(idx_cfg)

    @staticmethod
    def _probe_fps_width(video_path: str) -> tuple:
        """Return (fps, frame_width) for a video, with sensible fallbacks."""
        cap = cv2.VideoCapture(video_path)
        fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
        width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
        cap.release()
        return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

    @staticmethod
    def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
        """Provider-reported exchange count, else a duration-based estimate.

        One canonical implementation ? the twins had drifted (wasb relied on
        ``int(None)`` raising into the fallback; same result, by accident).
        """
        shot_count = segment.get("shuttle_exchange_count")
        if shot_count is None:
            return max(3, int(duration / 0.8))
        try:
            return int(shot_count)
        except (ValueError, TypeError):
            return max(3, int(duration / 0.8))

    # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged) -------
    def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                frame_width: float, video_path: str,
                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
        """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE
        returns exactly the 4 motion keys, so the trajectory twins persist byte-identical
        rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
        features). ``points`` are the whole-video trajectory points (TrajectoryPoint)."""
        return {
            "peak_velocity": cw["peak_velocity"],
            "mean_velocity": cw["mean_velocity"],
            "active_frame_count": cw["active_frame_count"],
            "track_id_count": cw["track_id_count"],
        }

    def _select_for_handoff(self, windows: List[Dict[str, Any]],
                            window_stats: List[Dict[str, Any]],
```

```
                            idx_cfg: Dict[str, Any]) -> List[int]:
        """Indices of the candidate windows that proceed to the AI handoff (and thus may
        become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
        ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
        Persisted candidates are NOT affected ? they remain the full superset for offline
        re-scoring."""
        return list(range(len(windows)))

    def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
                      player_pool: Optional[List[str]] = None,
                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
        # override-ignore: the ABC declares the Result contract (Success|Failure)
        # but every live segmenter still returns a bare list ? the documented
        # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
        label = self.display_label
        # --- Sport profile + AI provider wiring (identical across segmenters) ---
        sport_name = self.config.get("sport", "badminton")
        try:
            sport_profile = sport_registry.get(sport_name)
        except KeyError as e:
            logger.error(str(e))
```

27. assistant (2026-06-22T09:48:54.762Z)

Let me continue reading trajectory_hybrid to see what it returns:

28. user (2026-06-22T09:48:57.294Z)

```
149-        except KeyError as e:
150-            logger.error(str(e))
151:            return []
152-
153-        idx_cfg = self.config.get("indexing", {})
--
175-                    + api_key_hint(api_key_env_var)
176-                )
177:                return []
178-            try:
179-                provider = provider_registry.get(provider_name, api_key=api_key, model_name=model_name)
180-            except KeyError as e:
181-                logger.error(str(e))
182:                return []
183-
184-        video_duration = get_video_duration(video_path)
185-        if video_duration <= 0:
186-            logger.error("Invalid video duration.")
187:            return []
188-        fps, frame_width = self._probe_fps_width(video_path)
189-
--
198-            # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of crashing.
199-            logger.error("%s: %s", label, e)
200:            return []
201-
202-        velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
--
229-        if not ok:
230-            logger.error("%s detector environment not ready: %s", label, msg)
231:            return []
232-        logger.info("%s detector env OK: %s", label, msg)
233-
--
```

```
240-        if not csv_path:
241-            logger.error("%s produced no trajectory; aborting.", label)
242:            return []
243-
244-        points = runner.parse_trajectory_csv(csv_path)
--
287-
288-        if not all_candidate_windows:
289:            return []
290-
291-        # --- Phase 3: AI provider handoff over padded candidate clips ---
--
323-                w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
324-                if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
325:                    return []
326-                prompt = sport_profile.get_prompt(chunk_duration=w_dur)
327-                segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
--
377-                logger.warning("Skipping invalid segment struct: %s", e)
378-
379:        return segments_data
```

29. assistant (2026-06-22T09:49:01.918Z)

Now let me check the manifest configuration to understand what segmenter is used in the nightly test:

30. user (2026-06-22T09:49:03.735Z)

```
backend\pipeline\segmenters\__init__.py:1:from backend.pipeline.segmenters.base import
segmenter_registry, BasePlaySegmenter
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\__init__.py-2-from
backend.pipeline.segmenters.motion import MotionPlaySegmenter
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\__init__.py-3-from
backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\__init__.py-4-from
backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\__init__.py-5-from
backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
backend\pipeline\segmenters\__init__.py:6:from backend.pipeline.segmenters.wasb_hybrid import
WasbHybridSegmenter
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\__init__.py-7-from
backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\__init__.py-8-
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\__init__.py-9-__all__ = [
backend\pipeline\segmenters\__init__.py:10:    "segmenter_registry",
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\__init__.py-11-
"BasePlaySegmenter",
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\__init__.py-12-
"MotionPlaySegmenter",
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\yolo_hybrid.py-22-from typing
import List, Dict, Any, Tuple
C:\Users\avidu\Projects\badminton-highlight-
```

```
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\yolo_hybrid.py-23-
backend\pipeline\segmenters\yolo_hybrid.py:24:from backend.pipeline.segmenters.base import
BasePlaySegmenter, segmenter_registry
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\yolo_hybrid.py-25-from
backend.pipeline.detectors.person_detector import PersonDetector, _DEFAULT_DETECTOR
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\yolo_hybrid.py-26-from
backend.utils.video import get_video_duration, extract_video_chunk
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\yolo_hybrid.py-436-
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\yolo_hybrid.py-437-# Register
the segmenter
backend\pipeline\segmenters\yolo_hybrid.py:438:segmenter_registry.register(HybridYoloSegmenter)
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py-4-from typing import
List, Dict, Any, Tuple
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py-5-import logging
backend\pipeline\segmenters\motion.py:6:from backend.pipeline.segmenters.base import
BasePlaySegmenter, segmenter_registry
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py-7-from
backend.results import Success, Failure  # standardized contract
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py-8-
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py-253-
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py-254-# Register the
segmenter
backend\pipeline\segmenters\motion.py:255:segmenter_registry.register(MotionPlaySegmenter)
--
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\nightly-
e2e\backend\pipeline\segmenters\trajectory_hybrid.py-1-"""Shared engine for trajectory-substrate
hybrid segmenters.
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\trajectory_hybrid.py-2-
backend\pipeline\segmenters\trajectory_hybrid.py:3:tracknet_hybrid and wasb_hybrid were copy-
paste twins ? CODE_MAP carried a
backend\pipeline\segmenters\trajectory_hybrid.py-4-standing footgun: "any P3/P4 fix must be
applied to BOTH". This base class is
backend\pipeline\segmenters\trajectory_hybrid.py-5-now the single home for the whole pipeline
shape:
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\trajectory_hybrid.py-16-
detection_params extras such as weights_path).
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\trajectory_hybrid.py-17-
backend\pipeline\segmenters\trajectory_hybrid.py:18:This base is deliberately NOT registered;
only concrete subclasses register.
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\trajectory_hybrid.py-19-"""
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\trajectory_hybrid.py-20-import
os
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\trajectory_hybrid.py-111-
return max(3, int(duration / 0.8))
```

```
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\trajectory_hybrid.py-112-
backend\pipeline\segmenters\trajectory_hybrid.py:113:    # --- Fusion seams (identity by default
? wasb_hybrid/tracknet_hybrid unchanged) -------
backend\pipeline\segmenters\trajectory_hybrid.py-114-    def _enrich_candidate_stats(self, cw:
Dict[str, Any], points: List[Any], fps: float,
backend\pipeline\segmenters\trajectory_hybrid.py-115-
frame_width: float, video_path: str,
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\wasb_hybrid.py-15-from typing
import Any, Dict, Tuple
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\wasb_hybrid.py-16-
backend\pipeline\segmenters\wasb_hybrid.py:17:from backend.pipeline.segmenters.base import
segmenter_registry
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\wasb_hybrid.py-18-from
backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\wasb_hybrid.py-19-from
backend.pipeline.detectors.base import DetectorRunner
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\wasb_hybrid.py-27-
@property
backend\pipeline\segmenters\wasb_hybrid.py-28-    def name(self) -> str:
backend\pipeline\segmenters\wasb_hybrid.py:29:        return "wasb_hybrid"
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\wasb_hybrid.py-30-
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\wasb_hybrid.py-31-
@property
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\wasb_hybrid.py-47-
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\wasb_hybrid.py-48-
backend\pipeline\segmenters\wasb_hybrid.py:49:segmenter_registry.register(WasbHybridSegmenter)
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\fusion_hybrid.py-2-
backend\pipeline\segmenters\fusion_hybrid.py-3-Shuttle stays PRIMARY: this reuses an existing
trajectory substrate (WASB by default,
backend\pipeline\segmenters\fusion_hybrid.py:4:TrackNet optional) to seed candidate windows
exactly as `wasb_hybrid`/`tracknet_hybrid`
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\fusion_hybrid.py-5-do ? it
subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
backend\pipeline\segmenters\fusion_hybrid.py-6-shuttle-seeded window via two overridable seams
the engine exposes:
--
backend\pipeline\segmenters\fusion_hybrid.py-22-Defaults reproduce the substrate exactly:
net_crossings/person features are persisted but
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\fusion_hybrid.py-23-nothing is
gated (thresholds 0, person OFF), so `FusionSegmenter` with default config seeds
backend\pipeline\segmenters\fusion_hybrid.py:24:and hands off identically to `wasb_hybrid`.
Registered as ``fusion_hybrid``, NOT the default
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\fusion_hybrid.py-25-segmenter
? it is opt-in. Promotion to default happens only on measured LOVO lift through the
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\fusion_hybrid.py-26-experiment
harness (EXPERIMENT_HARNESS.md), never silently.
```

```
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\fusion_hybrid.py-28-from
typing import Any, Dict, List, Optional, Tuple
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\fusion_hybrid.py-29-
backend\pipeline\segmenters\fusion_hybrid.py:30:from backend.pipeline.segmenters.base import
segmenter_registry
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\fusion_hybrid.py-31-from
backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\fusion_hybrid.py-32-from
backend.pipeline.detectors.base import DetectorRunner
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\fusion_hybrid.py-187-
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\fusion_hybrid.py-188-
backend\pipeline\segmenters\fusion_hybrid.py:189:segmenter_registry.register(FusionSegmenter)
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\gemini.py-5-import
concurrent.futures
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\gemini.py-6-from typing import
List, Dict, Any, Tuple, Callable
backend\pipeline\segmenters\gemini.py:7:from backend.pipeline.segmenters.base import
BasePlaySegmenter, segmenter_registry
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\gemini.py-8-from
backend.utils.video import get_video_duration, extract_video_chunk
backend\pipeline\segmenters\gemini.py-9-from backend.utils.ffmpeg import ffmpeg_bin  # P2a:
single ffmpeg/ffprobe resolver
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\gemini.py-568-
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\gemini.py-569-# Register the
segmenter
backend\pipeline\segmenters\gemini.py:570:segmenter_registry.register(GeminiPlaySegmenter)
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\tracknet_hybrid.py-13-from
typing import Any, Dict, Tuple
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\tracknet_hybrid.py-14-
backend\pipeline\segmenters\tracknet_hybrid.py:15:from backend.pipeline.segmenters.base import
segmenter_registry
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\tracknet_hybrid.py-16-from
backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\tracknet_hybrid.py-17-from
backend.pipeline.detectors.base import DetectorRunner
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\tracknet_hybrid.py-40-
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\tracknet_hybrid.py-41-
backend\pipeline\segmenters\tracknet_hybrid.py:42:segmenter_registry.register(TrackNetHybridSegm
enter)
--
backend\pipeline\segmenters\base.py-37-        self._segmenters: Dict[str,
Type[BasePlaySegmenter]] = {}
```

```
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\base.py-38-
backend\pipeline\segmenters\base.py:39:    def register(self, segmenter_cls:
Type[BasePlaySegmenter]):
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\base.py-40-
"""Registers a new segmenter class under its name property."""
backend\pipeline\segmenters\base.py-41-        try:
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\base.py-50-
backend\pipeline\segmenters\base.py-51-    def list_available(self) -> Dict[str, str]:
backend\pipeline\segmenters\base.py:52:        """Returns a dict of registered segmenter names
and their descriptions."""
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\base.py-53-        res = {}
backend\pipeline\segmenters\base.py-54-        for name, cls in self._segmenters.items():
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\base.py-61-        return res
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\base.py-62-
backend\pipeline\segmenters\base.py:63:segmenter_registry = SegmenterRegistry()
```

31. assistant (2026-06-22T09:49:06.206Z)

Perfect. Now let me read the wasb_hybrid class to see if it properly wraps its return value:

32. user (2026-06-22T09:49:06.868Z)

```
"""WASB-based hybrid play segmenter.

Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
badminton weights and runs today, so this is the live shuttle-motion substrate.

For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
min_window_duration) are what get tuned against golden rally labels.

The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
WASB runner and identity.
"""
from typing import Any, Dict, Tuple

from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
from backend.pipeline.detectors.base import DetectorRunner
from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig


class WasbHybridSegmenter(TrajectoryHybridSegmenter):
    family = "wasb"
    display_label = "WASB"

    @property
    def name(self) -> str:
        return "wasb_hybrid"

    @property
    def description(self) -> str:
        return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
                "weights) for precise candidate rallies + AI provider for semantic
```

```
boundaries.")

    def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        cfg = WasbConfig.from_indexing_cfg(idx_cfg)
        return WasbRunner(cfg), {"weights_path": cfg.weights_path}

    def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        # Lazy import: the native runner pulls no heavy deps, but keep parity with the stub
        # seam (imported only when detector_impl='native' is actually selected).
        from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner, NativeWasbConfig
        cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
        return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
                                       "detector_impl": "native", "device": cfg.device}


segmenter_registry.register(WasbHybridSegmenter)

```

33. assistant (2026-06-22T09:49:10.045Z)

Now I have a critical understanding. Let me verify what motion.py returns vs what wasb_hybrid (via trajectory_hybrid) returns. The finding claims that line 349 in nightly_reelcount.py properly handles both Success and bare lists. Let me verify this claim:

34. user (2026-06-22T09:49:11.033Z)

```
    def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
        """
        Processes a video to detect play segments using motion heuristics,
        populates SQLite with detected segments, and indexes detailed events.
        """
        if not player_pool:
            player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]

        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            logger.error(f"Segmenter could not open video: {video_path}")
            return Failure(message=f"Could not open video: {video_path}", is_recoverable=False)

        final_segments = self._detect_motion_segments(cap, max_frames)

        segments_data = self._populate_db_with_events(video_id, final_segments, player_pool)

        return Success(value=segments_data)

    def _detect_motion_segments(self, cap: Any, max_frames: int = None) -> List[Tuple[float, float]]:
        """Run the pixel-motion heuristic over ``cap`` and return the detected
        (start, end) play segments.

        Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
        thresholds it into raw segments, then merges dead-time gaps and filters
        out segments shorter than ``min_segment_duration``. ``cap`` is released
        before returning. This is the detection half of ``process_video`` ? it
        performs no DB writes and no fabrication.
        """
        fps = cap.get(cv2.CAP_PROP_FPS)
```

35. assistant (2026-06-22T09:49:23.064Z)

Perfect! Now I have all the evidence. Let me summarize my findings:

CRITICAL FINDING - The claim is REAL but INCOMPLETE:

1. **motion.py** (line 37): Returns `Success(value=segments_data)` ? wrapped correctly.

2. **wasb_hybrid.py** (subclass of TrajectoryHybridSegmenter):
   - `wasb_hybrid.process_video` is inherited from `TrajectoryHybridSegmenter.process_video`
   - At **trajectory_hybrid.py line 138-140**, the override-ignore comment explicitly states: "the ABC declares the Result contract (Success|Failure) but every live segmenter still returns a bare list ? the documented deliberate-incremental gap (CODE_MAP gotchas; main.py handles both)."
   - Returns bare `List[Dict]` (e.g., line 379: `return segments_data`)

3. **nightly_reelcount.py line 349**: The code `segs = result.value if hasattr(result, "value") else result` does properly handle both:
   - `Success[list]` ? accesses `.value`
   - Bare `list` ? uses the list directly
   - But **it does NOT handle `Failure`** ? line 347-348 checks `isinstance(result, Failure)` first and returns early

The actual contract violation:
- The base class (`base.py` line 24) declares `Result[List[Dict]]` as the return type
- `motion.py` correctly returns `Success | Failure`
- `wasb_hybrid` (via `trajectory_hybrid`) returns bare `List[Dict]` despite the declared contract
- The comment at `trajectory_hybrid.py:141-143` documents this as "deliberate-incremental gap"

Line 349's defensive code works correctly for the current hybrid segmenters, but it masks a real contract violation. The handling is not a "proper contract adherence" ? it's a **workaround for non-compliant implementations**.

Let me call StructuredOutput with the findings:

36. user (2026-06-22T09:49:35.603Z)

Structured output provided successfully

37. assistant (2026-06-22T09:49:41.799Z)

Summary

The finding is REAL but represents the inverse of what was claimed:

The code at nightly_reelcount.py:349 does **work correctly** and handles both Success and bare list returns. However, this is not "proper contract adherence" ? it's a **documented workaround** for an intentional gap in the Result contract migration.

Evidence:
- **motion.py** correctly wraps returns in `Success[List[Dict]]` (line 37)
- **wasb_hybrid/trajectory_hybrid** return bare `List[Dict]` despite declaring `Result[...]` in the base class (base.py:24)
- **trajectory_hybrid.py:141-143** explicitly documents this as a "deliberate-incremental gap (CODE_MAP gotchas; main.py handles both)"
- **nightly_reelcount.py:349** is a defensive fallback that masks this contract violation ? the `hasattr` check allows it to tolerate both wrapped and unwrapped returns

Severity: Medium ? The code works reliably for the nightly test, but signals unresolved technical debt in the segmenter contract standardization (noted in results.py:3 as "low-regret item #3").